

Variable Scope



[_https://github.com/learn-co-curriculum/python-p3-variable-scope](https://github.com/learn-co-curriculum/python-p3-variable-scope)



[_https://github.com/learn-co-curriculum/python-p3-variable-scope/issues/new](https://github.com/learn-co-curriculum/python-p3-variable-scope/issues/new)

Learning Goals

- Understand the differences between variable scope in Python and JavaScript.
- Use local variables and global variables.

Key Vocab

- **Interpreter:** a program that executes other programs. Python programs require the Python interpreter to be installed on your computer so that they can be run.
- **Python Shell:** an interactive interpreter that can be accessed from the command line.
- **Data Type:** a specific kind of data. The Python interpreter uses these types to determine which actions can be performed on different data items.
- **Exception:** a type of error that can be predicted and handled without causing a program to crash.
- **Code Block:** a collection of code that is interpreted together. Python groups code blocks by indentation level.
- **Function:** a named code block that performs a sequence of actions when it is called.
- **Scope:** the area in your program where a specific variable can be called.

Introduction

In this lesson, we'll introduce the concept of scope in Python.

Variable Names and Scope

Naming variables can be challenging, but it is important. We need our code to be as descriptive as possible. Any other developer reading over one of our programs should be able to understand what our code does. A big part of this is using names that are sensible and descriptive.

What is Scope?

Functions in Python create their own scope. "Scope" refers to the areas of your program in which certain data is available to you. **Any variable created outside of a function will be considered *global* and made available to all functions in your module. Any variable created inside of your function (including parameters!) is considered *local* and only available inside of the function.**

Let's take a look at the following example:

```
name = "Joe"

def greeting():
    print(f"Hello, {name}")
```

In this code snippet, we have a variable, `name`, set equal to a string, `"Joe"`. Then we define a function, `greeting()`, that takes no arguments and prints out the value of the `name` variable. Let's fire up the Python shell, cross our fingers, and give it a go:

```
> greeting()
# Hello, Joe
```

As we can see, the value of our *global* variable was accessible inside of our `greeting()` function. Let's try another example:

```
name = "Joe"

def greeting(name):
    print(f"Hello, {name}")
```

In this code snippet, we have a variable, `name`, set equal to a string, `"Joe"`. Then we define a function, `greeting()`, that takes in an argument of name. Are the `name` variable that is set equal to `"Joe"` and the `name` variable we are using as a parameter (or argument) for our `greeting()` function definition the same? Let's see.

If we call our greeting function in the following way:

```
greeting("Sophie")
```

What do we expect to be outputted to the terminal?

In this case, the above function invocation would `print()` out `Hello, Sophie` to the terminal. It is true that we are setting a variable, `name`, equal to `"Joe"` in this code snippet. But we are not using *that* variable anywhere else in our code. The `name` argument of the `greeting()` function is just a placeholder. It means: when we call the `greeting()` function with an argument of, say, `"Sophie"`, set the variable `name` *inside of the function* equal to that string argument.

The `name` variable inside of the `greeting()` function is different from the `name` variable that we set equal to `"Joe"` outside of the function. The `greeting()` function has its own scope, and variables inside of it don't know about variables outside of it and vice versa.

Function Scope in Python

Just as in Python's global scope, JavaScript's outer scope rules would allow us to use a variable in the outer scope in our function:

```
const evilMonster = "Bowser";

function princessPeachsCastle() {
  console.log(`${evilMonster} is trying to kidnap Princess Peach!`);
}
```

And in Python:

```
evil_monster = "Bowser"

def princess_peachs_castle():
    print(f"{evil_monster} is trying to kidnap Princess Peach!")
```

So far, we've been looking from the *global* scope inward to the *local* scope. Let's take a look in the other direction:

If we define the following function to include a local variable:

```
def practicing_function_scope():
    im_trapped_in_the_function = "You can't access me outside this function!"
```

Trying to access that variable elsewhere in our program, *outside of this function*, will raise the following error:

```
> print(im_trapped_in_the_function)
# NameError: name 'im_trapped_in_the_function' is not defined
```

The **global** Keyword

Earlier, we accessed some *global* variables, `name` and `evil_monster`, from inside of functions. What do you think would happen if we tried to change a *global* value from a *local* scope?

```
change_the_world = "change yourself"

def change_yourself():
    change_the_world = "world changed"
```

When we enter this code in the Python shell and try to run `change_yourself()`, we don't see the change we're seeking:

```
> change_yourself()
> print(change_the_world)
```

```
# change yourself
```

To change the value of a *global* variable from a *local* scope, we need to use the `global` keyword:

```
change_the_world = "change yourself"

def change_yourself():
    global change_the_world
    change_the_world = "world changed"
```

Now when we run our function again, we should see the desired output:

```
> change_yourself()
> print(change_the_world)
# world changed
```

NOTE: It is **very rare** that you will use the `global` keyword in your code; Python's scope rules help provide predictable behavior and easier paths to follow backward to find bugs. Only override those rules when absolutely necessary!

Conclusion

Remember: A local variable defined inside a function can't leave that function. It is not available to your program outside of the function. A global variable defined outside of a function is accessible inside the function, just as in JavaScript. Global variables can only be *modified* from the global scope or by using the `global` keyword.